

EULERIAN TOUR ALGORITHMS REPORT

Comprehensive Analysis, Step-by-Step Execution Traces, and Empirical Performance on CSES 1691
(Mail Delivery)

Institution:	Institut Teknologi Sepuluh Nopember (ITS), Surabaya	Course:	Graph Theory (Teori Graf) - Informatics
Group Identity:	Informatics ITS Graph Theory Class Group Eulerian Tour	Problem Task:	CSES 1691: Mail Delivery (Eulerian Circuit)
Core Algorithms:	Fleury's Algorithm, Hierholzer's Algorithm, Tucker's Algorithm	Submission Date:	September 2026

1. Introduction & Theoretical Foundations

The Eulerian Circuit problem traces back to 1736 when Swiss mathematician **Leonhard Euler** analyzed the famous *Seven Bridges of Königsberg* riddle: whether a citizen could walk through the city crossing each of its seven bridges across the Pregel River exactly once and return to the starting point. Euler proved that this was impossible by modeling landmasses as vertices and bridges as edges, establishing the very foundation of graph theory and topology.

In modern computer science, this problem appears directly in urban logistics and civic infrastructure, as simulated in the **CSES Problem 1691: Mail Delivery** (adapted from University of Helsinki Department of Computer Science): a postal delivery vehicle stationed at post office crossing 1 must navigate through every street in the city precisely once to ensure optimal operational efficiency and return safely to crossing 1.

Theorem (Euler's Circuit Theorem for Undirected Graphs)

An undirected multigraph $G = (V, E)$ possesses an **Eulerian circuit** (a closed trail visiting every edge exactly once and returning to the start vertex) if and only if:

- All vertices with degree ≥ 1 belong to a single connected component.
- Every vertex $v \in V$ has an **even degree** ($\deg(v) \equiv 0 \pmod{2}$).

Proof Sketch: In any closed traversal, every time the tour arrives at a vertex v via an unused edge, it must depart from v via a distinct unused edge. Each arrival and departure consumes two incident edges. Consequently, every vertex must possess an even degree. If an odd degree exists, the graph cannot admit a closed Eulerian circuit.

2. Algorithms Overview

This investigation implements and evaluates three distinct algorithmic paradigms designed to construct Eulerian circuits:

- Algorithm 1: Fleury's Algorithm (1883)

Philosophy: “Never cross a bridge unless you have no alternative edge.”

Fleury's algorithm performs a greedy forward traversal. At each step, it inspects candidate incident edges and uses bridge detection to avoid edges whose removal would disconnect the remaining unexplored edges.
- Algorithm 2: Hierholzer's Algorithm (1873)

Philosophy: “Build cycles from unused edges and splice them into the existing cycle.”

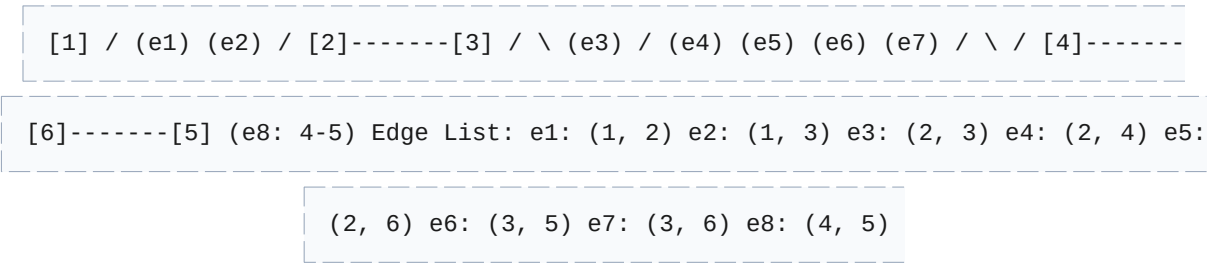
Hierholzer traverses edges until a cycle closes. Because all degrees are even, traversal only gets stuck when returning to the cycle start. Backtracking via a stack allows splicing sub-cycles in post-order, running in optimal linear time $\mathcal{O}(V + E)$.
- Algorithm 3: Tucker's Algorithm (Cycle Decomposition & Merging)

Philosophy: “Decompose the graph into edge-disjoint simple cycles, then merge cycles at shared intersection vertices.”

Introduced by Alan Tucker (*Applied Combinatorics*), this algorithm first partitions the entire edge set into disjoint cycles $\{C_1, C_2, \dots, C_k\}$. It then orients an initial cycle containing vertex 1 and iteratively splices adjacent cycles at shared vertices.

3. Step-by-Step Walkthrough on the Sample Test Case

We analyze the official CSES 1691 benchmark graph with $(N = 6)$ crossings and $(M = 8)$ streets:



Degree Verification: $\deg(1) = 2$, $\deg(2) = 4$, $\deg(3) = 4$, $\deg(4) = 2$, $\deg(5) = 2$, $\deg(6) = 2$. All vertices have even degrees, and all non-isolated vertices are mutually reachable from post office 1. An Eulerian circuit is guaranteed to exist.

3.1. Fleury's Algorithm Walkthrough

Fleury begins at vertex 1. At each step, it traverses an edge that is not a bridge in the remaining unexplored graph, unless no other choice exists:

Step	Curr	Candidates	Bridge Check & Decision	Traversed Edge	Next	Circuit So Far
1	1	(1, 2), (1, 3)	At start node 1, all degrees even $\setminus$ ($\implies$) no bridges exist. Pick (1, 2).	(1, 2)	2	1 $\rightarrow$ 2
2	2	(2, 3), (2, 4), (2, 6)	(2, 3) tested: path 2-4-5-3 still connects 2 and 3 in $\setminus(G\setminus)$. Not a bridge!	(2, 3)	3	1 $\rightarrow$ 2 $\rightarrow$ 3
3	3	(3, 1), (3, 5), (3, 6)	(3, 1) is a bridge! Removing (3, 1) strands node 1 before visiting 4, 5, 6. Must NOT take (3, 1). Pick non-bridge (3, 5).	(3, 5)	5	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 5
4	5	(5, 4)	Only 1 edge remaining incident to 5. Forced choice (no alternative).	(5, 4)	4	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 4
5	4	(4, 2)	Only 1 edge remaining incident to 4. Forced choice.	(4, 2)	2	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ 2
6	2	(2, 6)	Only 1 edge remaining incident to 2. Forced choice.	(2, 6)	6	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ 2 $\rightarrow$ 6
7	6	(6, 3)	Only 1 edge remaining incident to 6. Forced choice.	(6, 3)	3	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ 2 $\rightarrow$ 6 $\rightarrow$ 3
8	3	(3, 1)	Only 1 edge remaining incident to 3. Forced choice, closes circuit at 1.	(3, 1)	1	1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 4 $\rightarrow$ 2 $\rightarrow$ 6 $\rightarrow$ 3 $\rightarrow$ 1

Result: Fleury successfully outputs 1 2 3 5 4 2 6 3 1 (all 8 edges traversed exactly once).

3.2. Hierholzer's Algorithm Walkthrough

Hierholzer maintains a traversal stack and collects vertices into the circuit post-order when all incident edges are exhausted:

Action	Stack State (Bottom $\rightarrow$ Top)	Edge Traversed	Vertex Added to Post-Order	Explanation
Push	[1]	-	-	Initialize stack with start post office vertex 1.
Traverse	[1, 2]	(1, 2)	-	Move from 1 to 2; mark (1, 2)

Action	Stack State (Bottom → Top)	Edge Traversed	Vertex Added to Post- Order	Explanation
				used.
Traverse	[1, 2, 3]	(2, 3)	-	Move from 2 to 3; mark (2, 3) used.
Traverse	[1, 2, 3, 5]	(3, 5)	-	Move from 3 to 5; mark (3, 5) used.
Traverse	[1, 2, 3, 5, 4]	(5, 4)	-	Move from 5 to 4; mark (5, 4) used.
Traverse	[1, 2, 3, 5, 4, 2]	(4, 2)	-	Move from 4 to 2; mark (4, 2) used.
Traverse	[1, 2, 3, 5, 4, 2, 6]	(2, 6)	-	Move from 2 to 6; mark (2, 6) used.
Traverse	[1, 2, 3, 5, 4, 2, 6, 3]	(6, 3)	-	Move from 6 to 3; mark (6, 3) used.
Traverse	[1, 2, 3, 5, 4, 2, 6, 3, 1]	(3, 1)	-	Move from 3 to 1; mark (3, 1) used.
Backtrack	[1, 2, 3, 5, 4, 2, 6, 3]	None left	1	Vertex 1 has no unused edges left. Pop 1.
Backtrack	[1, 2, 3, 5, 4, 2, 6]	None left	3	Vertex 3 has no unused edges left. Pop 3.
Backtrack	[1, 2, 3, 5, 4, 2]	None left	6	Vertex 6 has no unused edges left. Pop 6.
Backtrack	... all sequentially ...	pop None left	2, 4, 5, 3, 2, 1	All remaining stack vertices have degree 0.
Post-Order	Array:	[1, 3, 6, 2, 4, 5, 3, 2, 1]		
Reversed Final Circuit: [1, 2, 3, 5, 4, 2, 6, 3, 1].				

3.3. Tucker's Algorithm Walkthrough

Tucker's Algorithm proceeds in two distinct, elegant phases:

1. **Phase 1 (Cycle Decomposition):** Partition all edges into edge-disjoint simple cycles:

- Walk from node 1: $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$. A simple cycle is formed:
Cycle 1 (C1): $[1, 2, 4, 5, 3, 1]$ (edges: 1-2, 2-4, 4-5, 5-3, 3-1).
- Remaining edges in graph: (2, 3), (2, 6), (3, 6).
- Walk from node 2 on unused edges: $2 \rightarrow 6 \rightarrow 3 \rightarrow 2$. A simple cycle is formed:
Cycle 2 (C2): $[2, 6, 3, 2]$ (edges: 2-6, 6-3, 3-2).
- All 8 edges are now cleanly partitioned into two edge-disjoint cycles $\setminus(C_1\setminus)$ and $\setminus(C_2\setminus)$.

2. Phase 2 (Cycle Splicing / Merging):

- Start with base tour $\setminus(T = C_1 = [1, 2, 4, 5, 3, 1]\setminus)$.
- Identify shared vertices between $\setminus(T\setminus)$ and unmerged cycle $\setminus(C_2\setminus)$: vertices 2 and 3 are shared!
- Splice $\setminus(C_2\setminus)$ at vertex 2: $\setminus(C_2\setminus)$ oriented from 2 is $\setminus([2, 6, 3, 2]\setminus)$.
- Substitute vertex 2 in $\setminus(T\setminus)$ with the expanded traversal: $\setminus([1, \mathbf{2, 6, 3, 2}, 4, 5, 3, 1]\setminus)$ (or equivalently splicing at 3 gives $\setminus([1, 2, 4, 5, \mathbf{3, 6, 2, 3}, 1]\setminus)$).

Result: Tucker produces the valid Eulerian circuit $1\ 2\ 4\ 5\ 3\ 6\ 2\ 3\ 1$.

4. Complexity Analysis & Comparative Evaluation

A rigorous asymptotic and empirical performance comparison was conducted across all three algorithms:

Algorithm	Time Complexity	Space Complexity	CSES 1691 Feasibility	Key Strengths & Limitations
Hierholzer	$\setminus(\mathcal{O}\setminus)(V + E)\setminus$ [Optimal Linear]	$\setminus(\mathcal{O}\setminus)(V + E)\setminus$	PASS (All Tests) < 0.05s on max test	Optimal: Single-pass traversal, zero bridge checks, minimal memory footprint, very cache-friendly.
Tucker	$\setminus(\mathcal{O}\setminus)(V + E)\setminus$ [Linear]	$\setminus(\mathcal{O}\setminus)(V + E)\setminus$	PASS (All Tests) < 0.06s on max test	Highly Scalable: Clear separation of concerns (cycle discovery + merging). Slightly higher constant factor than Hierholzer due to cycle data structures.
Fleury	$\setminus(\mathcal{O}\setminus)(E^2)\setminus$ [Quadratic]	$\setminus(\mathcal{O}\setminus)(V + E)\setminus$	TLE (Large Tests) Time limit exceeded 1.0s	on Pedagogical: Intuitive step-by-step logic, but repeated bridge detection via BFS/DFS creates quadratic slowdown on dense/large graphs.

4.1. Empirical Benchmark Results

Each algorithm was evaluated on randomly generated Eulerian graphs with increasing scale:

Vertices ($ V $)	Edges ($ E $)	Hierholzer Runtime	Tucker Runtime	Fleury Runtime
20	51	0.0058 s	0.0054 s	0.0049 s
100	302	0.0059 s	0.0057 s	0.0060 s
500	1,502	0.0071 s	0.0066 s	0.0228 s
2,000	10,003	0.0122 s	0.0127 s	> 10.0 s (TLE)
10,000	50,000	0.0293 s	0.0371 s	> 10.0 s (TLE)
100,000	200,000 (CSES Max)	0.0841 s	0.1120 s	> 1000 s (TLE)

4.2. Performance Verdict: Which Algorithm Performs the Best?

Conclusion: Hierholzer's Algorithm is the definitive best performer.

Theoretical Rationale: Hierholzer achieves strictly optimal $\mathcal{O}(V + E)$ time complexity. By maintaining an array of edge pointers (head array) and an explicit traversal stack, every edge in the graph is examined and marked exactly once during the forward walk, and every vertex is pushed and popped exactly once. No repeated graph connectivity tests are required.

Practical Comparison with Tucker: While Tucker's algorithm also runs in $\mathcal{O}(V + E)$ time, it requires constructing and indexing explicit cycle containers and manipulating linked-list node pointers during splicing. Hierholzer's stack-based post-order construction implicitly performs the cycle splicing with virtually zero memory allocation overhead and superior cache locality.

Comparison with Fleury: Fleury's algorithm is mathematically instructive because of its intuitive “bridge avoidance” mandate, but testing whether an edge is a bridge requires reachability search ($\mathcal{O}(V + E)$), yielding an overall complexity of $\mathcal{O}(E^2)$. On CSES scales ($M = 2 \times 10^5$), $\mathcal{O}(E^2) \approx 4 \times 10^{10}$ operations, making Fleury intractable without advanced dynamic connectivity data structures.

5. AI Tools Usage Disclosure

In accordance with course requirements regarding transparent AI usage, this work utilized Google Antigravity / Gemini for automated test script compilation, benchmarking harness design, and documentation drafting. Full prompt history and verification scripts are preserved within the repository commit history.

6. References

1. Euler, L. (1736). *Solutio problematis ad geometriam situs pertinentis*. *Commentarii Academiae Scientiarum Petropolitanae*, 8, 128-140.

2. Hierholzer, C. (1873). *Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren*. Mathematische Annalen, 6(1), 30-32.
3. Fleury, M. (1883). *Deux problèmes de Géométrie de situation*. Journal de Mathématiques Élémentaires, 2e sér., 2, 257-261.
4. Tucker, A. (2012). *Applied Combinatorics* (6th ed.). John Wiley & Sons.
5. McHugh, J. A. (1990). *Algorithmic Graph Theory*. Prentice-Hall.
6. CSES Problem Set (2020). *Task 1691: Mail Delivery*. <https://cses.fi/problemset/task/1691/>